Project Planning & Management: Smart Vision System

1. Project Proposal

1.1 Overview

Under the Digital Egypt Pioneers Initiative (DEPI), this project proposes the development of a “Smart Vision System for Small Businesses,” an AI-driven Computer Vision Software-as-a-Service (SaaS) platform. The core innovation lies in transforming standard, pre-existing CCTV cameras into intelligent, real-time video analytics hubs without necessitating expensive proprietary hardware upgrades. By leveraging advanced deep learning architectures and modern web technologies, the system will provide small businesses with enterprise-grade security and operational insights.

1.2 Objectives

- **Cost-Effectiveness:** Eliminate the need for specialized AI hardware by processing video streams from standard IP/CCTV cameras.
- **Real-Time Analytics:** Provide instantaneous insights, including foot traffic counting and zone monitoring, with minimal latency.
- **Enhanced Security:** Implement proactive threat detection (e.g., weapon detection) coupled with smart alerting mechanisms.
- **Operational Intelligence:** Generate heatmaps and time-based behavioral analyses to optimize business layouts and staffing.
- **Scalable Architecture:** Design a modular SaaS framework capable of supporting multi-camera deployments in future commercial iterations.

1.3 Scope

The project encompasses the complete software development lifecycle of the Smart Vision System, including:

- AI model training, fine-tuning, and optimization for specific use cases (person detection, weapon detection).

- Implementation of multi-object tracking algorithms (ByteTrack) for unique ID assignment.
- Development of a robust RESTful API and WebSocket infrastructure for real-time data streaming.
- Design and development of an interactive frontend dashboard using Next.js to visualize analytics, heatmaps, and alerts.
- Deployment of the Minimum Viable Product (MVP) using simulated pre-recorded video environments to validate system accuracy and performance.

1.4 Out of Scope

To strictly manage stakeholder expectations and ensure compliance with legal and ethical standards, the following features are explicitly **Out of Scope** for this graduation project:

- **Facial Recognition & Personal Identity Logging:** The system will strictly detect "persons" as bounding boxes and will not biometrically identify individuals, ensuring compliance with data privacy laws (e.g., GDPR and local Egyptian data protection regulations).
- **Automated Physical Hardware Control:** The system is restricted to software-based smart alerts and dashboard notifications. It will not integrate with or trigger physical hardware mechanisms such as electronic door locks, automated turnstiles, or physical sirens.
- **Audio Analysis:** The platform is a purely vision-based analytics system. It will not process, record, or analyze audio feeds from cameras.

---

2. Task Assignment & Roles

Team Member	Designated Role	Technical Responsibilities	Documentation & Management Responsibilities
Mohamed Wageh Fathy Abo Seleem	Project Leader & System Architect	High-level system architecture, database schema design, API contract definition, technology stack finalization, and code review.	Project plan drafting, Gantt chart maintenance, risk management, progress tracking, and final project report compilation.

Mohamed El Metwaly Ahmed abdelbary	AI Core Team (Lead Data & Model)	Data collection, annotation pipeline setup, YOLOv8 model training, hyperparameter tuning, and model evaluation (mAP validation).	AI model selection justification, dataset documentation, and model performance evaluation reports.
Alaa Abouelnaga Hassan	AI Core Team (Tracking & Logic)	Integration of ByteTrack algorithm, object ID assignment logic, entry/exit line-crossing logic, and heatmap data generation.	Documentation of tracking algorithms, zone logic flowcharts, and API payload structures for AI outputs.
Ibrahim Yasser Mohammed	AI Core Team (Specialized Models)	Fine-tuning specialized models (e.g., weapon detection), time-based behavior analysis logic implementation, and false-positive reduction.	Documentation of behavior analysis rules, weapon detection thresholds, and testing protocols for edge cases.
Rawan tahseen fouad	Backend & API Developer	FastAPI implementation, WebSocket configuration for real-time streaming, video ingestion pipeline, and cloud deployment.	API documentation (Swagger/OpenAPI), backend deployment guides, and server configuration documentation.
Mohamed Reda Sadek Mohamed	Frontend & UI/UX Developer	Next.js dashboard development, WebSocket client integration, rendering heatmaps/tracks on canvas, and responsive UI design.	UI/UX wireframes, user manual creation, and frontend component library documentation.

---

3. Project Plan & Timeline (Gantt Chart Representation)

Note: Timeline commences Mid-April 2026 and strictly adheres to the July 17, 2026, final deadline.

Phase	Key Tasks & Deliverables	Start Date	End Date	Duration	Milestone Status
Phase 1: Requirements & Design	SRS finalization, UI/UX wireframes, System Architecture design, Dataset identification.	Apr 15, 2026	Apr 28, 2026	2 Weeks	Baseline Sign-off
Phase 2: AI Model Development	Data annotation, YOLOv8 training (Persons/Weapons), ByteTrack integration, Heatmap logic.	Apr 29, 2026	May 31, 2026	4.5 Weeks	AI Model Freeze
Phase 3: Backend & API Development	FastAPI setup, Video ingestion (4-5 min test video), WebSocket streaming, AI-Backend integration.	Jun 01, 2026	Jun 22, 2026	3 Weeks	API Operational
Phase 4: Frontend & Dashboard UI	Next.js setup, Real-time data visualization, Zone management UI, Alert system interface.	Jun 10, 2026	Jul 01, 2026	3 Weeks	UI Feature Complete
Phase 5: Integration & System Testing	End-to-end integration, latency optimization, false-positive tuning, bug fixing.	Jul 02, 2026	Jul 10, 2026	1.5 Weeks	**Code Implementation Deadline**

Phase 6: Final Presentation Prep	Final deck creation, demo environment stabilization, mock defense Q&A.	Jul 11, 2026	Jul 16, 2026	6 Days	Presentation Ready
Phase 7: Final Delivery	Project submission, live demonstration, and official DEPI evaluation.	Jul 17, 2026	Jul 17, 2026	1 Day	**Final Presentation Deadline**

---

4. Resource Allocation

4.1 Hardware Resources

- **Input Source:** Standard CCTV/IP cameras.
- **Testing Environment:** A 4-5 minute pre-recorded, high-definition video simulating retail foot traffic and edge-case scenarios (e.g., weapon presence, loitering) will be used to validate the pipeline prior to live camera integration.
- **Development Machines:** Team-specific workstations equipped with dedicated GPUs (e.g., NVIDIA RTX series) required solely for the local AI model training phase.

4.2 Software Tools & Technologies

- **AI & Tracking Stack:** Python, PyTorch, Ultralytics (YOLOv8), ByteTrack, OpenCV.
- **Backend Stack:** FastAPI, Uvicorn, WebSockets, Python-dotenv.
- **Frontend Stack:** Next.js, React, Tailwind CSS, HTML5 Canvas (for heatmap rendering), Socket.io-client.
- **Version Control & CI/CD:** Git, GitHub, GitHub Actions (for automated linting and basic testing).

4.3 Cloud Deployment Strategy

- **MVP Phase (Current Project):** Free-tier cloud services (Render or Railway for backend hosting, Vercel for Next.js frontend) utilizing CPU-only instances. The system will process the pre-recorded video frame-by-frame to simulate real-time stream processing, validating the SaaS logic without incurring GPU cloud costs.
- **Future Commercial Phase:** Migration to scalable GPU-backed infrastructure (e.g., AWS EC2 G4dn instances or Google Cloud Compute Engine with attached GPUs) to handle concurrent, live multi-camera RTSP streams.

5. Risk Assessment & Mitigation Plan

Risk ID	Risk Description	Probability	Impact	Mitigation Strategy
R01	**Poor Video Resolution & Lighting Conditions:** Standard CCTV cameras may produce grainy or overexposed footage, degrading AI accuracy.	High	High	Implement aggressive image augmentation techniques (blur, noise, brightness variation) during the YOLOv8 training phase. Define strict minimum hardware guidelines for camera placement in user documentation.
R02	**High Server Latency During Real-Time Processing:** Processing high-framerate video on CPU-based free-tier cloud servers may cause severe lag and frame dropping.	High	High	Implement frame-sampling logic (e.g., processing every 3rd frame) to reduce computational load. Ensure asynchronous processing pipelines in FastAPI to prevent thread blocking.
R03	**AI Model Overfitting to Test Data:** The model may perform exceptionally well on the 4-5 minute test video but fail in generic real-world environments.	Medium	High	Utilize a diverse training dataset (merging COCO, Roboflow public datasets, and custom data). Employ cross-validation techniques and monitor validation loss to apply early stopping during training.
R04	**WebSocket Connection Instability:** Continuous real-time data streaming between backend and frontend may drop due	Medium	Medium	Implement robust auto-reconnect logic with exponential backoff on the Next.js client. Add heartbeat/ping mechanisms to keep the free-tier server

	to free-tier cloud idling or network fluctuations.			awake during active sessions.

---

6. KPIs (Key Performance Indicators)

To objectively measure the success and readiness of the Smart Vision System MVP, the following quantitative KPIs have been established:

- **AI Detection Accuracy (mAP@0.5):** Achieve a Mean Average Precision of **> 90%** for person detection and **> 85%** for weapon detection on the validation dataset.
- **API & Processing Latency:** Maintain an end-to-end frame processing and WebSocket broadcast latency of **< 200 milliseconds** per sampled frame under standard CPU cloud deployment.
- **Tracking Robustness (ID Switch Rate):** Maintain an ID switch rate of **< 5%** during the 4-5 minute test video to ensure reliable entry/exit counting and heatmap generation.
- **System Uptime & Stability:** The MVP dashboard and API must sustain **99% availability** during the final 72-hour pre-presentation testing window without manual intervention or server crashes.
- **Frontend Rendering Performance:** The Next.js dashboard must successfully render real-time bounding boxes and update the heatmap canvas at a minimum of **15 Frames Per Second (FPS)** without UI freezing or memory leaks.